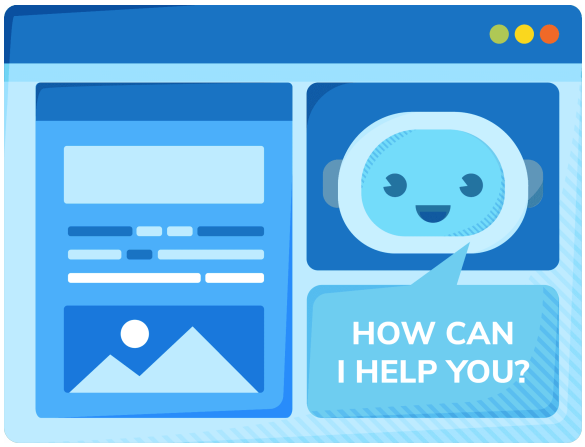
Demo Application

Explore how to build a Flask application that uses OpenAI's chat completions endpoint. Learn to set up routes, design prompts, and fetch responses to create interactive NLP-powered web apps using Python.

In this lesson, we'll present an example Flask application to integrate the chat completions endpoint of OpenAI API.



Demo application

Workflow

Let's first look at the workflow of the application:

1. When we run the application, the home page appears. The home page has two sections with buttons to navigate the other pages – "Chat compl and "Specific Tasks."

Ask

2. The “Completions” button takes us to the page to design our own prompt and get results from the OpenAI API. We need to select the model and enter the prompt. We can then click the “Submit” button to fetch the response from the completions API.
3. Similarly, the “Specific Tasks” button takes us to the page where we can enter text and receive results from the OpenAI API. We need to select both the model and the operation and enter the prompt. Then, we can click the “Submit” button to fetch the response from the completions API.

The demo application can be launched by clicking “Run” in the playground below. Next, click the URL next to “Your app can be found at:” to interact with the application.

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

```
from flask import Flask, request, render_template,jsonify
from openai import OpenAI

app = Flask(__name__)

# Set up OpenAI API key
# Please replace {{SECRET_KEY}} with your own API key.
client = OpenAI(api_key="{{SECRET_KEY}}")

# List of available models
models = ["gpt-3.5-turbo", "gpt-3.5-turbo-0125", "gpt-3.5-turbo-1106"]

@app.route("/results", methods=["GET"])
def results():
    if request.method == "GET":
        # Get the selected model and prompt from the form
        model = request.args.get('model')
        prompt = request.args.get('prompt')

        # Call the OpenAI API with the selected model and prompt
```

Ask

```
        response = client.chat.completions.create(
            model=model,
            messages=[
                {"role": "user", "content": prompt}
            ]
        )
        # result = response['choices'][0]['message']['content']
        result = response.choices[0].message.content

        # Render the results page with the response
        return render_template("results.html", response=result)
```

Flask demo application

Code explanation

In this explanation, we'll focus on dissecting the `app.py` file, which is the main script responsible for handling the logic of our application. Here is the line by line explanation of the code in the `app.py` file.

- **Lines 1–2:** The code imports necessary modules from the Flask framework and the OpenAI API.
- **Line 4:** An instance of the Flask class is created and assigned to the variable, `app`.
- **Line 8:** The OpenAI API key is set up using the `OpenAI` class from the `openai` module. Replace `{{SECRET_KEY}}` with the actual API key.
- **Line 11:** A list named `models` is defined, containing three available models.
- **Lines 13–34:** The `/results` route is defined to handle `GET` requests. When this route is accessed via a `GET` request, it retrieves the selected model and prompt from the form, calls the OpenAI API with the provided model and prompt, renders the results page with the response, and returns it.

Ask

- **Lines 36–60:** The `/results_new` route is similar to `/results` but also includes the operation. It retrieves the selected model, prompt, and operation from the form, calls the OpenAI API with the provided model, prompt, and operation, renders the results page with the response, model, prompt, and operation, and returns it.
- **Lines 62–64:** The `/` route is the home page route, which renders the `index.html` template.
- **Lines 66–68:** The `/prompt_design` route renders the `prompt_design.html` template.
- **Lines 70–72:** The `/specific_tasks` route renders the `specific_tasks.html` template.
- **Line 75:** The `app.run()` method is called to run the Flask application.

In conclusion, this lesson has provided a practical demonstration of how Flask seamlessly integrates with OpenAI's API, offering a straightforward approach to prompt design and result retrieval.

Ask